

南開大學
Nankai University

计算机学院

计算机组成原理 **lab5** 实验报告

单周期 CPU 实现

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

2026 年 5 月 27 日

目录

1 实验目的 **2**

2 实验内容 **2**

3 实验原理图 **3**

3.1 单周期 CPU 实现原理图 3

3.2 单周期 CPU 顶层模块原理图 4

3.3 汇编示意图 4

4 实验步骤 **6**

4.1 `single_cycle_cpu.v` 的修改 6

4.2 `inst_rom.v` 的修改 7

4.3 `tb.v` 的修改 7

5 实验结果分析 **8**

5.1 仿真结果 8

5.2 上板验证结果 9

5.3 结果分析 11

6 实验问题与解决 **11**

6.1 `bgtz` 偏移量修正 11

6.2 下载线驱动问题 12

7 总结感想 **12**

A 源代码 **12**

A.1 `single_cycle_cpu.v` 12

A.2 `inst_rom.v` 18

A.3 `tb.v` 20

1 实验目的

本实验在实验指导手册中单周期 CPU 实验的基础上进行功能复现与改进。通过阅读和理解原有单周期 CPU 的 Verilog 代码，掌握 CPU 中取指、译码、执行、访存和写回等基本阶段的实现方法，并进一步理解指令译码、控制信号生成、寄存器堆读写、ALU 运算以及分支跳转之间的关系。

本次实验的主要目的如下：

1. 理解单周期 CPU 的整体结构和基本工作流程，掌握 PC、指令存储器、寄存器堆、ALU、数据存储器以及写回通路之间的连接关系。
2. 在原有单周期 CPU 支持指令的基础上，新增两条 MIPS 指令：`sub` 和 `bgtz`。其中，`sub` 为 R 型减法指令，`bgtz` 为 I 型大于零分支跳转指令。
3. 掌握 R 型指令和 I 型指令的指令格式、机器码编码方式以及在 CPU 中的译码方法，能够根据指令的 `op`、`funct`、`rs`、`rt` 等字段生成相应控制信号。
4. 修改 `inst_rom.v` 文件，在原有测试程序之后加入斐波那契数列计算程序，使用已有指令和新增指令共同完成 `Fib(10)` 的计算。

2 实验内容

本次实验以实验指导手册中提供的单周期 CPU 工程为基础，对原有 CPU 进行功能扩展。原 CPU 已经能够支持若干条基本 MIPS 指令，并能够通过 LCD 显示当前 PC、当前指令、寄存器内容和数据存储器内容。本实验在此基础上主要完成新增 `sub` 指令、新增 `bgtz` 指令，以及修改 `inst_rom.v` 实现 `Fib(10)` 计算三部分内容。

▶ 新增 R 型指令 `sub`

对 `single_cycle_cpu.v` 进行修改，增加 R 型减法指令 `sub`。该指令功能为：

$$rd \leftarrow rs - rt$$

其操作码 `op` 为 `000000`，功能码 `funct` 为 `100010`。由于原 CPU 已经支持 `subu` 指令和 ALU 减法通路，因此 `sub` 可以复用已有减法硬件，只需要补充指令译码、ALU 控制信号和写回控制，使运算结果能够写入 R 型指令的目标寄存器 `rd`。

▶ 新增 I 型指令 `bgtz`

增加 I 型条件分支指令 `bgtz`，其功能为：

$$\text{if } rs > 0 \text{ then branch}$$

该指令操作码 `op` 为 `000111`，且 `rt` 字段固定为 `00000`。对于 32 位补码数，`rs > 0` 可判断为符号位为 0 且数据不为 0，因此本实验使用 `(~rs_value[31]) & (|rs_value)` 作为跳转条件。当条件成立时，CPU 选择分支目标地址作为下一条指令地址。

▶ 修改 `inst_rom.v` 实现 `Fib(10)`

修改 inst_rom.v，在原有测试程序之后插入斐波那契数列计算程序。根据实验要求，原程序 00H--48H 部分保持不变，将原来的 j 00H 后移到新增程序之后。新增程序使用 $13 保存当前项 a，使用 $14 保存下一项 b，使用 $15 作为循环计数器，使用 $16 保存临时结果，使用 $17 保存常数 1。

程序通过循环计算得到：

$$Fib(10) = 55 = 0x00000037$$

其中，新增的 sub 用于循环计数器递减，新增的 bgtz 用于判断循环是否继续。循环结束后执行：

$$sw\ \$14,\ 32(\$0)$$

将最终结果写入数据存储器地址 20H。因此，实验最终应在仿真波形和实验箱 LCD 上观察到：

$$REG0E = 0x00000037, \qquad Mem[20H] = 0x00000037$$

其中 REG0E 即寄存器 $14，用于保存最终的 Fib(10) 结果。

3 实验原理图

3.1 单周期 CPU 实现原理图

单周期 CPU 的实现框图如图 3.1 所示。CPU 主要由 PC、指令存储器 Inst_rom、控制器 Control unit、寄存器堆 Regfile、符号扩展模块、ALU、数据存储器 Data_ram 以及若干多路选择器组成。

本次实验在原有数据通路基础上增加了 sub 和 bgtz 两条指令的支持。其中，sub 指令复用 ALU 中已有的减法通路；bgtz 指令通过判断 rs_value > 0 产生 bgtz_taken 信号，并与 beq_taken、bne_taken 等分支信号共同参与下一条 PC 的选择。

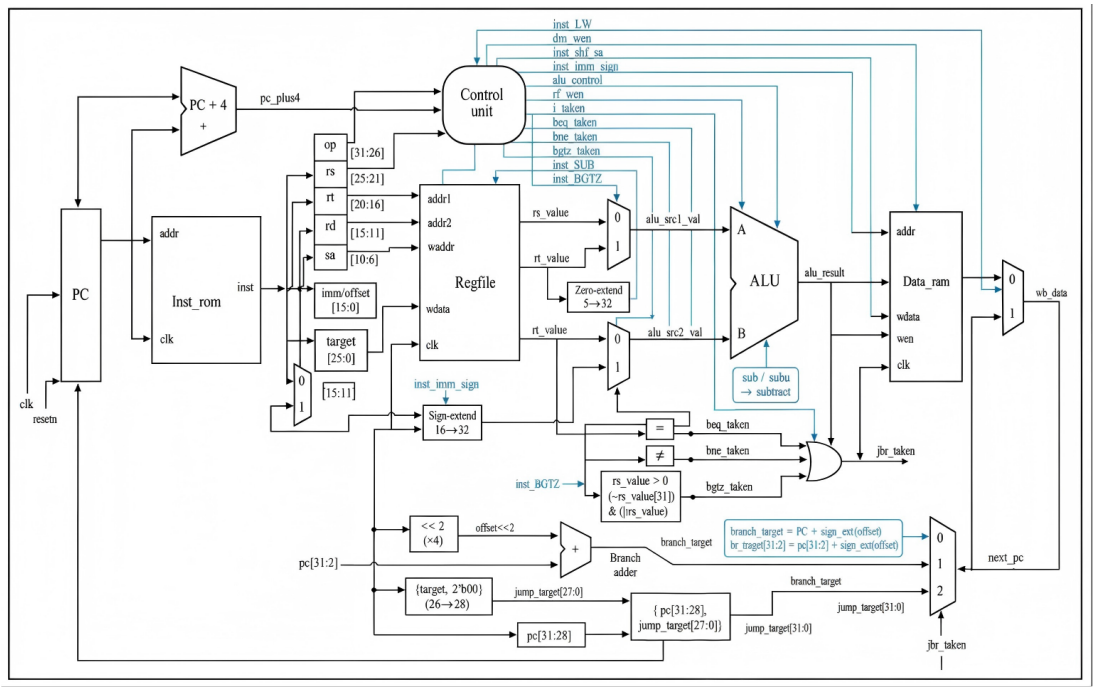


图 3.1: 单周期 CPU 实现原理图

需要注意的是，本实验中分支目标地址的计算基于当前 PC，而不是标准 MIPS 中的 PC+4。因此 bgtz 指令从 6CH 跳回 58H 时，偏移量按照当前 PC 计算，最终使用的机器码为 1DE0FFFB。

3.2 单周期 CPU 顶层模块原理图

单周期 CPU 顶层模块原理图如图 3.2 所示。顶层模块为 single_cycle_cpu_display.v，主要包含 single_cycle_cpu.v、lcd_module、单步时钟生成模块、输入地址寄存器、寄存器地址生成模块以及显示选择逻辑。

其中，按键时钟 btn_clk 经过 single_step_clk_gen 后生成 cpu_clk，作为 CPU 的单步运行时钟。lcd_module 输出 input_valid、input_value 和 display_number，用于输入观察地址和选择 LCD 显示内容。CPU 输出的 PC、INST、test_rf_data 和 test_mem_data 经过显示选择逻辑后送回 LCD 模块显示。

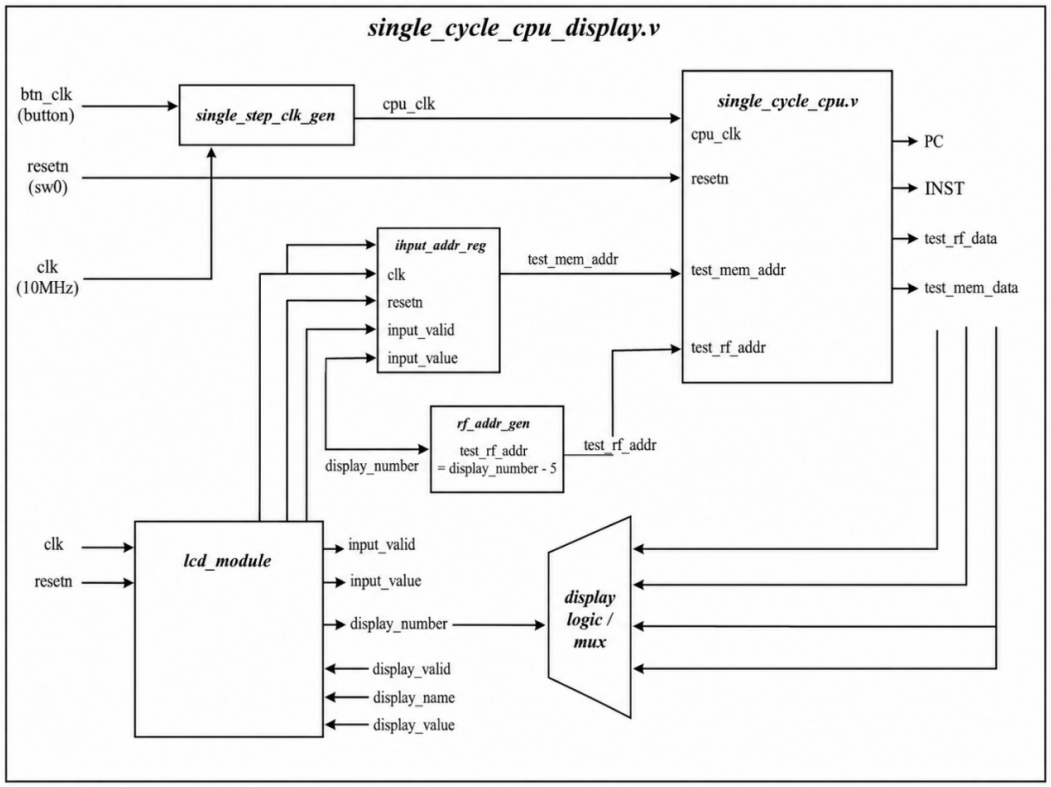


图 3.2: 单周期 CPU 顶层模块原理图

3.3 汇编示意图

Fib(10) 汇编程序示意图如图 3.3 所示。程序在原有 00H--48H 测试程序之后执行，首先初始化寄存器 $13、$14 和 $15，然后进入循环体。循环中使用 addu 完成斐波那契数列相邻两项相加，使用新增的 sub 指令实现计数器递减，使用新增的 bgtz 指令判断是否继续循环。

循环结束后，程序执行：

$$sw\ \$14,\ 32(\$0)$$

将最终结果写入数据存储器地址 20H。最终应得到：

$$REG0E = 00000037, \qquad Mem[20H] = 00000037$$

即：

$$Fib(10) = 55 = 0x00000037$$

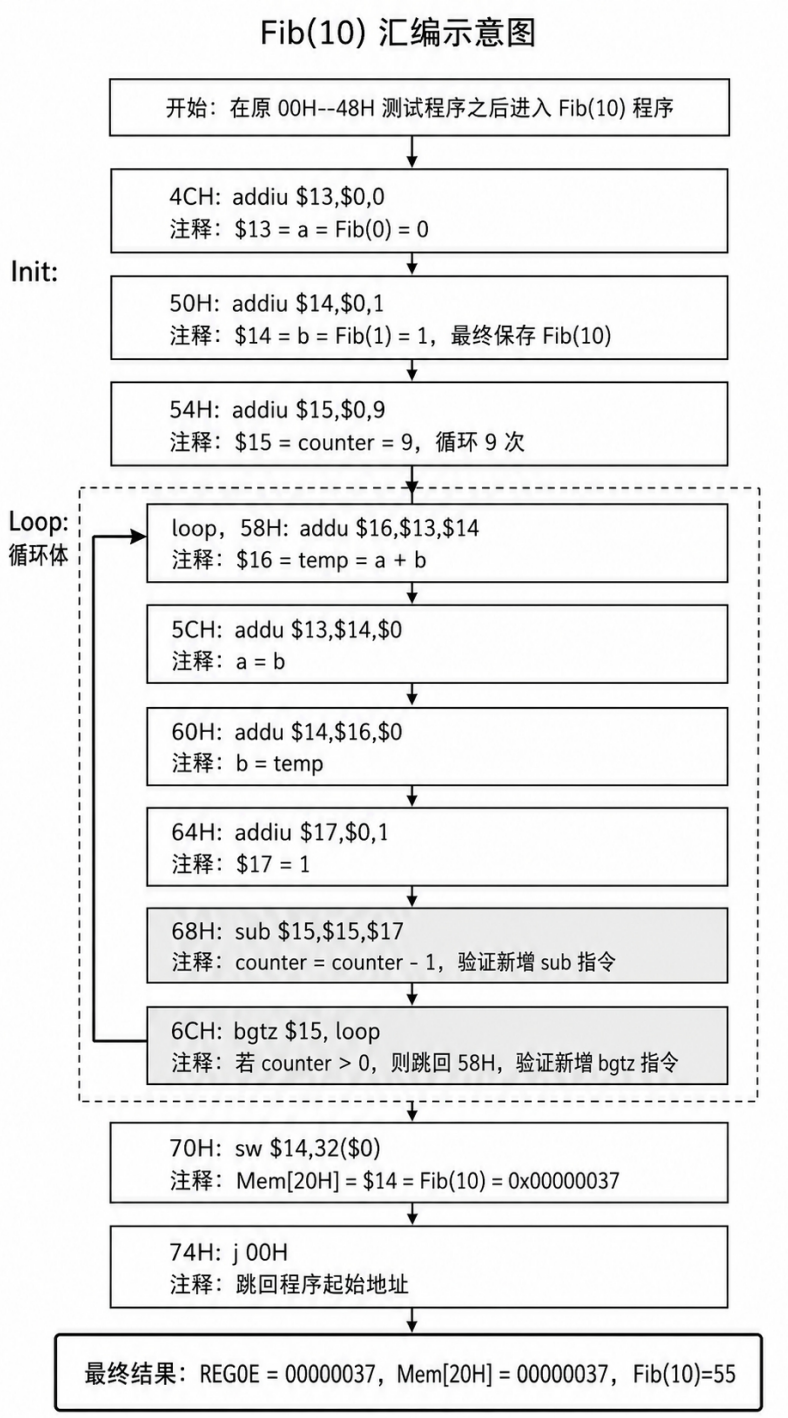


图 3.3: Fib(10) 汇编程序示意图

4 实验步骤

本次实验主要完成三个源文件的修改：首先在 single_cycle_cpu.v 中增加 sub 和 bgtz 两条指令的译码与控制逻辑；然后在 inst_rom.v 中扩展指令存储器内容，加入 Fib(10) 测试程序；最后修改 tb.v，便于在仿真中直接观察寄存器 $14 和数据存储器 20H 的最终结果。

4.1 single_cycle_cpu.v 的修改

single_cycle_cpu.v 是本实验中 CPU 核心控制逻辑所在文件，主要负责指令译码、ALU 控制、访存控制、寄存器写回控制以及 PC 更新控制。本次实验在该文件中增加了 sub 和 bgtz 两条指令的支持。

▶ 增加 sub 指令译码

sub 是 R 型指令，其操作码 op 为 000000，功能码 funct 为 100010。因此在译码部分新增 inst_SUB 信号，用于识别当前指令是否为 sub。

```
assign inst_SUB = op_zero & sa_zero & (funct == 6'b100010); // 【新增】sub指令译码

```

由于 sub 与原有 subu 一样都执行减法操作，因此在 ALU 控制部分将 inst_SUB 并入减法控制信号中，使其复用原有减法通路。

```
assign inst_sub = inst_SUBU | inst_SUB; // 【修改】sub复用原有减法通路

```

同时，sub 是 R 型写寄存器指令，目标寄存器为 rd，因此需要将其加入写回目标选择逻辑。

```
assign inst_wdest_rd = inst_ADDU | inst_SUBU | inst_SUB | ... ; // 【修改】sub结果写入rd

```

▶ 增加 bgtz 指令译码与跳转条件

bgtz 是 I 型条件分支指令，其操作码 op 为 000111，并且 rt 字段固定为 00000。因此在译码部分新增 inst_BGTZ 信号。

```
assign inst_BGTZ = (op == 6'b000111) & (rt == 5'd0); // 【新增】bgtz指令译码

```

对于 32 位补码数，大于 0 的判断条件为符号位为 0 且数据不为 0，因此新增 bgtz_taken 信号。

```
assign bgtz_taken = (~rs_value[31]) & (|rs_value); // 【新增】rs > 0 时跳转

```

最后，将 bgtz 的跳转条件加入 PC 更新控制逻辑。当 bgtz 条件成立时，下一条指令地址选择分支目标地址。

```
assign jbr_taken = j_taken
                 | inst_BEQ  & beq_taken
```

```
                | inst_BNE  & bne_taken
                | inst_BGTZ & bgtz_taken; // 【修改】加入bgtz跳转控制

```

4.2 inst_rom.v 的修改

inst_rom.v 是 CPU 的指令存储器文件，用于根据当前 PC 地址输出对应的机器指令。本次实验需要在原有测试程序之后加入 Fib(10) 程序，因此首先将指令 ROM 的容量从原来的 20 条扩展为 30 条。

```
wire [31:0] inst_rom[29:0]; // 【修改】扩展到30条指令，用于加入Fib(10)程序
```

原程序的 00H--48H 部分保持不变，在其后插入新增的斐波那契数列计算程序，并将原来位于 4CH 的 j 00H 移动到新增程序末尾。新增程序的主要思路是：使用 $13 保存当前项 a，使用 $14 保存下一项 b，使用 $15 作为循环计数器，循环 9 次后得到 Fib(10)。

新增指令序列如下：

```
assign inst_rom[19] = 32'h240D0000; // 4CH: addiu $13,$0,#0    | a = 0
assign inst_rom[20] = 32'h240E0001; // 50H: addiu $14,$0,#1    | b = 1
assign inst_rom[21] = 32'h240F0009; // 54H: addiu $15,$0,#9    | counter = 9

assign inst_rom[22] = 32'h01AE8021; // 58H: addu  $16,$13,$14  | temp = a + b
assign inst_rom[23] = 32'h01C06821; // 5CH: addu  $13,$14,$0   | a = b
assign inst_rom[24] = 32'h02007021; // 60H: addu  $14,$16,$0   | b = temp
assign inst_rom[25] = 32'h24110001; // 64H: addiu $17,$0,#1    | $17 = 1
assign inst_rom[26] = 32'h01F17822; // 68H: sub   $15,$15,$17  | counter--
assign inst_rom[27] = 32'h1DE0FFFB; // 6CH: bgtz  $15,loop     | 若counter>0跳回58H
assign inst_rom[28] = 32'hAC0E0020; // 70H: sw    $14,#32($0)  | Mem[20H] = Fib(10)
assign inst_rom[29] = 32'h08000000; // 74H: j     00H          | 跳回起始地址
```

其中，sub 指令用于实现循环计数器递减，bgtz 指令用于判断是否继续循环。需要注意的是，本实验 CPU 的分支目标地址计算方式是基于当前 PC 加偏移量，而不是标准 MIPS 中的 PC+4 加偏移量。因此从 6CH 跳回 58H 时，偏移量为：

$$\frac{0x58 - 0x6C}{4} = -5 = 0xFFFB$$

所以 bgtz 的机器码应为 1DE0FFFB。

最后，在 case 语句中补充 5'd19 到 5'd29 的读取逻辑，保证新增指令能够被正确取出。

4.3 tb.v 的修改

tb.v 是行为级仿真的测试文件，用于产生时钟、复位信号，并设置寄存器堆和数据存储器的调试读取地址。为了便于直接观察 Fib(10) 的结果，本实验将寄存器调试地址设置为 0E，即寄存器 $14；将数据存储器调试地址设置为 00000020。

```
initial begin
    clk = 0;
```

```
    resetn = 0;

    rf_addr = 5'd14;              // 【修改】观察寄存器$14，Fib(10)最后保存在$14中
    mem_addr = 32'h00000020;      // 【修改】观察Mem[20H]，Fib(10)会写到该地址

    #100;
    resetn = 1;
end
```

这样在仿真波形中，只需要观察 `rf_data` 和 `mem_data` 即可判断程序是否执行正确。若 CPU 正确完成 `Fib(10)` 计算，则仿真最终应出现：

$$rf_data = 00000037, \qquad mem_data = 00000037$$

其中，`0x00000037` 即十进制 55，对应 `Fib(10)` 的正确结果。

5 实验结果分析

5.1 仿真结果

仿真时，将寄存器堆调试地址设置为 `0E`，即观察寄存器 `$14`；同时将数据存储器调试地址设置为 `00000020`，即观察保存 `Fib(10)` 结果的数据存储器地址。

仿真波形如图 5.4 所示。

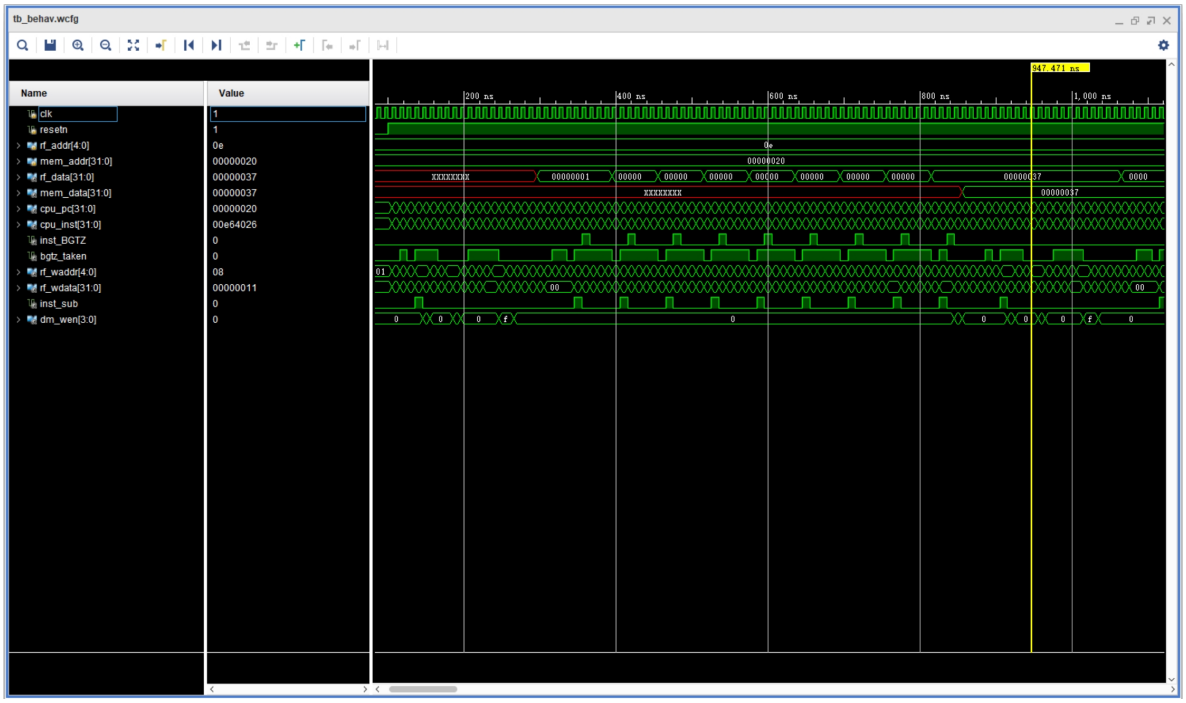


图 5.4: Vivado 行为级仿真结果

由图 5.4 可以看到，仿真结果中：

$$rf_addr = 0e, \qquad rf_data = 00000037$$

说明寄存器 `$14` 中保存的值为 `0x00000037`。同时，波形中还可以观察到：

$$mem_addr = 00000020, \qquad mem_data = 00000037$$

说明数据存储器地址 `20H` 中也保存了 `0x00000037`。由于：

$$0x00000037 = 55$$

因此仿真结果表明，CPU 能够正确执行新增的斐波那契数列程序，并计算得到 `Fib(10)=55`。

5.2 上板验证结果

行为级仿真通过后，继续完成综合、实现和 bitstream 生成，并通过 Vivado Hardware Manager 将设计下载到 FPGA 实验箱。上板验证时，在 LCD 上输入内存观察地址：

$$MADDR = 00000020$$

然后通过单步运行 CPU，观察当前 PC、当前指令、寄存器值和数据存储器值。
图 5.5 为程序运行到 `bgtz` 指令时的上板结果。

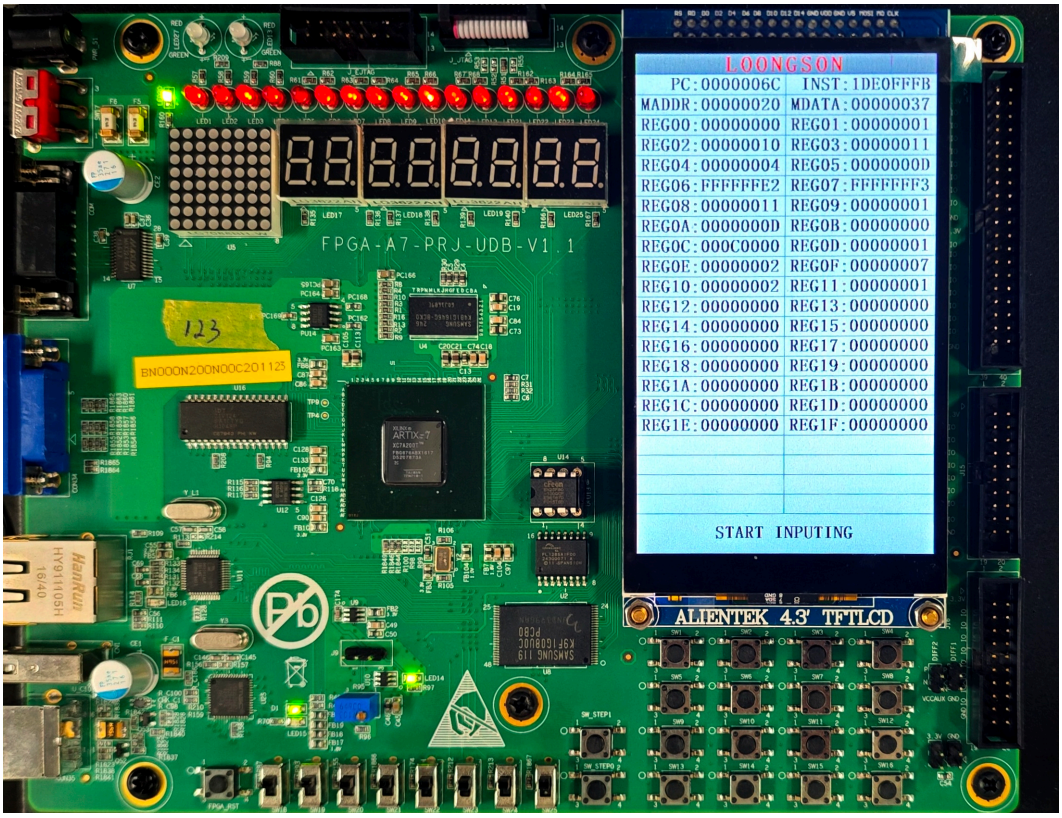


图 5.5: 上板执行 `bgtz` 指令时的结果

由图 5.5 可以看到，LCD 显示：

$$PC = 0000006C, \qquad INST = 1DE0FFFB$$

其中，机器码 `1DE0FFFB` 对应本实验新增的 `bgtz` 指令：

$$bgtz\ \$15,\ loop$$

该指令用于判断循环计数器 `$15` 是否大于 0，若条件成立，则跳回循环入口继续计算。因此，该图可以说明 CPU 已经能够正确取到并执行新增的 `bgtz` 指令。

图 5.6 为程序执行到最终写内存指令时的上板结果。

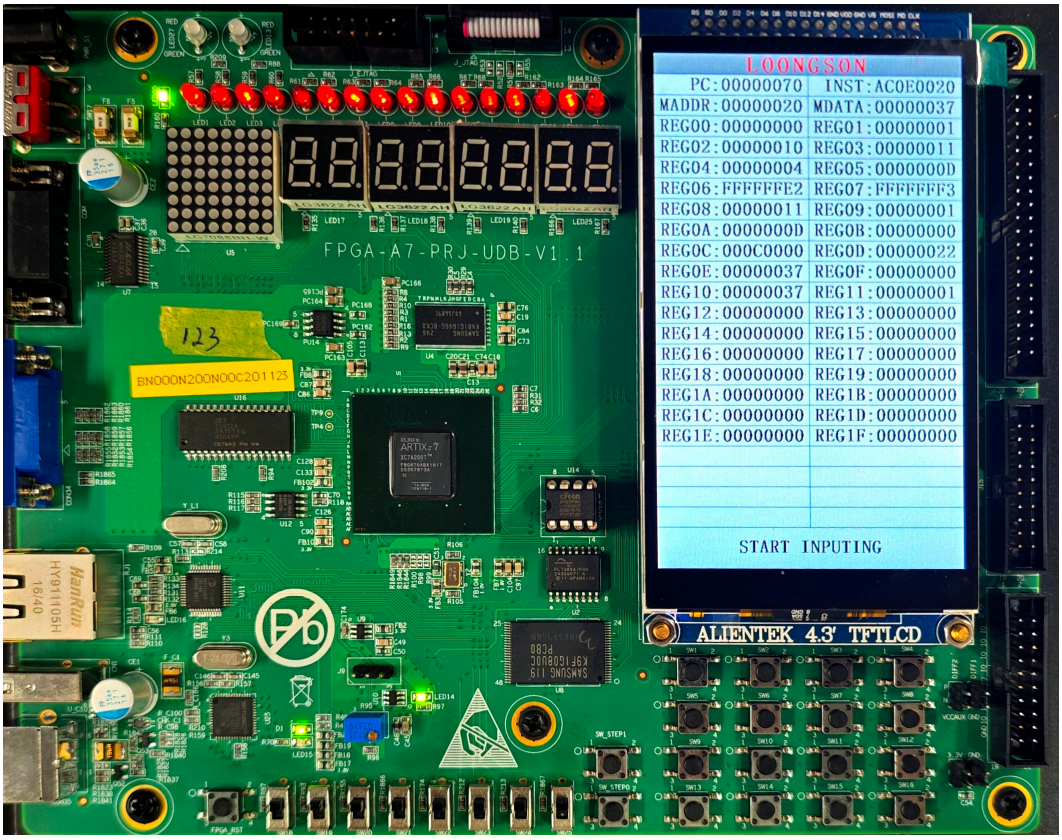


图 5.6: 上板最终结果验证

由图 5.6 可以看到，LCD 显示：

$$PC = 00000070, \qquad INST = AC0E0020$$

其中，机器码 `AC0E0020` 对应指令：

$$sw\ \$14,\ 32(\$0)$$

该指令将寄存器 `$14` 中的结果写入数据存储器地址 `20H`。同时，LCD 上还显示：

$$MADDR = 00000020, \qquad MDATA = 00000037$$

并且：

$$REG0E = 00000037$$

其中，REG0E 即寄存器 $14。这说明上板运行时，寄存器 $14 和数据存储器 20H 中均得到了 0x00000037，与仿真结果一致。

5.3 结果分析

综合仿真结果和上板结果可以看出，本次实验对单周期 CPU 的功能扩展是正确的。

首先，新增的 sub 指令在斐波那契程序中用于循环计数器递减：

$$\$15 \leftarrow \$15 - \$17$$

如果 sub 指令不能正确执行，则循环计数器无法从 9 逐步递减到 0，程序无法正确完成固定次数的循环，最终也无法得到正确的 Fib(10) 结果。

其次，新增的 bgtz 指令用于控制循环是否继续执行。当 $15 > 0 时，程序跳回循环入口；当 $15 = 0 时，跳转条件不成立，程序继续向下执行写内存指令。上板结果中可以观察到 PC=0000006C 时执行的指令为 1DE0FFFB，说明 CPU 能够正确取到新增的 bgtz 指令。

最后，仿真波形和实验箱 LCD 均显示：

$$REG0E = 00000037, \qquad Mem[20H] = 00000037$$

即：

$$Fib(10) = 55 = 0x00000037$$

这说明新增的 sub 和 bgtz 指令能够配合原有指令正确执行斐波那契数列程序，实验结果满足要求。

6 实验问题与解决

在本次实验过程中，除了完成代码修改和功能验证外，还遇到了一些仿真和上板调试问题。通过逐步分析问题现象、定位原因并修改工程配置或代码，最终完成了仿真验证和实验箱上板验证。

6.1 bgtz 偏移量修正

在最初编写 Fib(10) 程序时，bgtz 指令的机器码设置为：

$$1DE0FFFA$$

仿真过程中发现，程序虽然能够执行到 bgtz 指令附近，但循环跳转效果不符合预期。进一步分析 single_cycle_cpu.v 中的分支目标地址计算逻辑后发现，本实验 CPU 的分支目标地址不是按照标准 MIPS 的 PC+4+offset<<2 方式计算，而是按照当前 PC 加偏移量计算：

```
1 assign br_target[31:2] = pc[31:2] + {{14{offset[15]}}, offset};
```

因此，从 6CH 跳回循环入口 58H 时，偏移量应为：

$$\frac{0x58 - 0x6C}{4} = -5$$

−5 的 16 位补码为 FFFB，因此 bgtz 指令应修改为：

$$1DE0FFFB$$

修改后，对应代码为：

```
assign inst_rom[27] = 32'h1DE0FFFB; // 【修改】bgtz跳回58H，offset=-5
```

重新仿真后，程序能够正确完成循环，并最终得到 Fib(10)=0x00000037。

6.2 下载线驱动问题

由于近期参加电赛，使用 Keil 和相关单片机调试器驱动，电脑的 USB 驱动环境可能发生了变化。

在生成 bitstream 后进行上板时，Vivado Hardware Manager 只能显示 localhost(0)，无法识别 FPGA 器件。检查设备管理器后发现，Xilinx Platform Cable USB Firmware Loader 发生 52 号报错，提示驱动数字签名无法验证。

我重新安装了 Vivado 自带的 cable driver，并在 Windows 高级启动中临时禁用驱动程序强制签名。之后重新尝试，完成了 bitstream 下载。

7 总结感想

通过本次单周期 CPU 改进实验，我进一步理解了 CPU 的整体数据通路和控制逻辑。相比之前单独实现寄存器堆、ALU 等模块，本次实验需要综合分析取指、译码、执行、访存和写回等环。

新增 sub 和 bgtz 指令的过程中，我进一步熟悉了 R 型指令和 I 型分支指令的指令格式、译码方法以及控制信号生成方式。尤其是在调试 bgtz 偏移量时，我认识到指令编码必须和 CPU 实际采用的分支地址计算方式保持一致，不能简单套用标准 MIPS 中 PC+4 的计算方法。

本次实验将前面实验中涉及的寄存器堆、ALU 等内容联系起来，使我对单周期 CPU 的整体结构和运行机制有了更加系统的理解。

附录 A 源代码

A.1 single_cycle_cpu.v

Listing 1: single_cycle_cpu.v

```
`timescale 1ns / 1ps

//*************************************************************
// > 文件名: single_cycle_cpu.v
// > 描述   : 单周期 CPU 模块，共实现 16 条指令
// >          指令 rom 和数据 ram 均采用异步读数据，以便单周期 CPU 好实现
// > 作者   : LOONGSON
```

```
8  // > 日期  : 2016-04-14
9  //*****************************************************************
10
11 `define STARTADDR 32'd0    // 程序起始地址
12
13 module single_cycle_cpu(
14     input clk,              // 时钟
15     input resetn,           // 复位信号，低电平有效
16
17     // display data
18     input  [ 4:0] rf_addr,
19     input  [31:0] mem_addr,
20     output [31:0] rf_data,
21     output [31:0] mem_data,
22     output [31:0] cpu_pc,
23     output [31:0] cpu_inst
24 );
25
26 //---------------------------{取指}begin----------------------//
27 reg [31:0] pc;
28 wire [31:0] next_pc;
29 wire [31:0] seq_pc;
30 wire [31:0] jbr_target;
31 wire jbr_taken;
32
33 // 下一指令地址：seq_pc = pc + 4
34 assign seq_pc[31:2] = pc[31:2] + 1'b1;
35 assign seq_pc[1:0]  = pc[1:0];
36
37 // 新指令：若指令跳转，为跳转地址；否则为下一指令
38 assign next_pc = jbr_taken ? jbr_target : seq_pc;
39
40 always @(posedge clk)       // PC 程序计数器
41 begin
42     if (!resetn)
43     begin
44         pc <= `STARTADDR;   // 复位，取程序起始地址
45     end
46     else
47     begin
48         pc <= next_pc;      // 不复位，取新指令
49     end
50 end
51
52 wire [31:0] inst_addr;
53 wire [31:0] inst;
54 assign inst_addr = pc;      // 指令地址：指令长度 32 位
55
56 inst_rom inst_rom_module(   // 指令存储器
```

```
57      .addr (inst_addr[6:2]), // I, 5, 指令地址
58      .inst (inst)            // O, 32, 指令
59  );
60
61  assign cpu_pc   = pc;       // display pc
62  assign cpu_inst = inst;
63  //-----------------------------{取指}end-----------------------//
64
65
66  //-----------------------------{译码}begin-----------------------//
67  wire [5:0] op;
68  wire [4:0] rs;
69  wire [4:0] rt;
70  wire [4:0] rd;
71  wire [4:0] sa;
72  wire [5:0] funct;
73  wire [15:0] imm;
74  wire [15:0] offset;
75  wire [25:0] target;
76
77  assign op     = inst[31:26];    // 操作码
78  assign rs     = inst[25:21];    // 源操作数 1
79  assign rt     = inst[20:16];    // 源操作数 2
80  assign rd     = inst[15:11];    // 目标操作数
81  assign sa     = inst[10:6];     // 特殊域，可能存放偏移量
82  assign funct  = inst[5:0];      // 功能码
83  assign imm    = inst[15:0];     // 立即数
84  assign offset = inst[15:0];     // 地址偏移量
85  assign target = inst[25:0];     // 目标地址
86
87  wire op_zero;                   // 操作码全 0
88  wire sa_zero;                   // sa 域全 0
89  assign op_zero = ~(|op);
90  assign sa_zero = ~(|sa);
91
92  // 实现指令列表
93  wire inst_ADDU, inst_SUBU, inst_SUB, inst_SLT, inst_AND; // 【新增】inst_SUB: R型有符号减法 sub
94  wire inst_NOR,  inst_OR,   inst_XOR, inst_SLL;
95  wire inst_SRL,  inst_ADDIU, inst_BEQ, inst_BNE, inst_BGTZ; // 【新增】inst_BGTZ: I型大于0跳转
96  wire inst_LW,   inst_SW,    inst_LUI, inst_J;
97
98  assign inst_ADDU  = op_zero & sa_zero & (funct == 6'b100001); // 无符号加法
99  assign inst_SUBU  = op_zero & sa_zero & (funct == 6'b100011); // 无符号减法
100 assign inst_SUB   = op_zero & sa_zero & (funct == 6'b100010); //
    【新增】sub: R型有符号减法，funct=100010
101 assign inst_SLT   = op_zero & sa_zero & (funct == 6'b101010); // 小于则置位
102 assign inst_AND   = op_zero & sa_zero & (funct == 6'b100100); // 逻辑与
103 assign inst_NOR   = op_zero & sa_zero & (funct == 6'b100111); // 逻辑或非
104 assign inst_OR    = op_zero & sa_zero & (funct == 6'b100101); // 逻辑或
```

```
105 assign inst_XOR   = op_zero & sa_zero & (funct == 6'b100110); // 逻辑异或
106 assign inst_SLL   = op_zero & (rs == 5'd0) & (funct == 6'b000000); // 逻辑左移
107 assign inst_SRL   = op_zero & (rs == 5'd0) & (funct == 6'b000010); // 逻辑右移
108 assign inst_ADDIU = (op == 6'b001001); // 立即数无符号加法
109 assign inst_BEQ   = (op == 6'b000100); // 判断相等跳转
110 assign inst_BNE   = (op == 6'b000101); // 判断不等跳转
111 assign inst_BGTZ  = (op == 6'b000111) & (rt == 5'd0); // 【新增】bgtz: rs>0时跳转, rt字段必须为0
112 assign inst_LW    = (op == 6'b100011); // 从内存装载
113 assign inst_SW    = (op == 6'b101011); // 向内存存储
114 assign inst_LUI   = (op == 6'b001111); // 立即数装载高半字节
115 assign inst_J     = (op == 6'b000010); // 直接跳转
116
117 // 无条件跳转判断
118 wire j_taken;
119 wire [31:0] j_target;
120 assign j_taken  = inst_J;
121 assign j_target = {pc[31:28], target, 2'b00};
122
123 // 分支跳转
124 wire beq_taken;
125 wire bne_taken;
126 wire bgtz_taken; // 【新增】bgtz跳转条件信号
127 wire [31:0] br_target;
128
129 assign beq_taken  = (rs_value == rt_value);       // BEQ 跳转条件: GPR[rs] = GPR[rt]
130 assign bne_taken  = ~beq_taken;                   // BNE 跳转条件: GPR[rs] != GPR[rt]
131 assign bgtz_taken = (~rs_value[31]) & (|rs_value); //
        【新增】BGTZ条件: rs为正数, 即符号位为0且不等于0
132 assign br_target[31:2] = pc[31:2] + {{14{offset[15]}}, offset};
133 assign br_target[1:0]  = pc[1:0];                 // 分支跳转目标地址: PC = PC + offset << 2
134
135 // 跳转指令的跳转信号和跳转目标地址
136 assign jbr_taken = j_taken                        // 指令跳转: 无条件跳转或满足分支跳转条件
137                  | inst_BEQ  & beq_taken
138                  | inst_BNE  & bne_taken
139                  | inst_BGTZ & bgtz_taken;        // 【新增】bgtz满足rs>0时跳转
140 assign jbr_target = j_taken ? j_target : br_target;
141
142 // 寄存器堆
143 wire rf_wen;
144 wire [4:0] rf_waddr;
145 wire [31:0] rf_wdata;
146 wire [31:0] rs_value, rt_value;
147
148 regfile rf_module(
149     .clk       (clk),        // I, 1
150     .wen       (rf_wen),     // I, 1
151     .raddr1    (rs),         // I, 5
152     .raddr2    (rt),         // I, 5
```

```
153     .waddr      (rf_waddr),   // I, 5
154     .wdata      (rf_wdata),   // I, 32
155     .rdata1     (rs_value),   // O, 32
156     .rdata2     (rt_value),   // O, 32
157 
158     // display rf
159     .test_addr (rf_addr),
160     .test_data (rf_data)
161 );
162 
163 // 传递到执行模块的 ALU 源操作数和操作码
164 wire inst_add, inst_sub, inst_slt, inst_sltu;
165 wire inst_and, inst_nor, inst_or, inst_xor;
166 wire inst_sll, inst_srl, inst_sra, inst_lui;
167 
168 assign inst_add  = inst_ADDU | inst_ADDIU | inst_LW | inst_SW; // 做加法运算
169 assign inst_sub  = inst_SUBU | inst_SUB; // 【修改】subu和新增sub都使用ALU减法
170 assign inst_slt  = inst_SLT;  // 小于置位
171 assign inst_sltu = 1'b0;      // 暂未实现
172 assign inst_and  = inst_AND;  // 逻辑与
173 assign inst_nor  = inst_NOR;  // 逻辑或非
174 assign inst_or   = inst_OR;   // 逻辑或
175 assign inst_xor  = inst_XOR;  // 逻辑异或
176 assign inst_sll  = inst_SLL;  // 逻辑左移
177 assign inst_srl  = inst_SRL;  // 逻辑右移
178 assign inst_sra  = 1'b0;      // 暂未实现
179 assign inst_lui  = inst_LUI;  // 立即数装载高位
180 
181 wire [31:0] sext_imm;
182 wire inst_shf_sa;             // 使用 sa 域作为偏移量的指令
183 wire inst_imm_sign;           // 对立即数作符号扩展的指令
184 
185 assign sext_imm      = {{16{imm[15]}}, imm}; // 立即数符号扩展
186 assign inst_shf_sa   = inst_SLL | inst_SRL;
187 assign inst_imm_sign = inst_ADDIU | inst_LUI | inst_LW | inst_SW;
188 
189 wire [31:0] alu_operand1;
190 wire [31:0] alu_operand2;
191 wire [11:0] alu_control;
192 
193 assign alu_operand1 = inst_shf_sa   ? {27'd0, sa} : rs_value;
194 assign alu_operand2 = inst_imm_sign ? sext_imm     : rt_value;
195 
196 assign alu_control = {inst_add,     // ALU 操作码, 独热编码
197                       inst_sub,
198                       inst_slt,
199                       inst_sltu,
200                       inst_and,
201                       inst_nor,
```

```
                        inst_or,
                        inst_xor,
                        inst_sll,
                        inst_srl,
                        inst_sra,
                        inst_lui};
//----------------------------{译码}end-------------------------//


//----------------------------{执行}begin-----------------------//
wire [31:0] alu_result;

alu alu_module(
    .alu_control (alu_control),  // I, 12, ALU 控制信号
    .alu_src1    (alu_operand1), // I, 32, ALU 操作数 1
    .alu_src2    (alu_operand2), // I, 32, ALU 操作数 2
    .alu_result  (alu_result)    // O, 32, ALU 结果
);
//----------------------------{执行}end-------------------------//


//----------------------------{访存}begin-----------------------//
wire [3 :0] dm_wen;
wire [31:0] dm_addr;
wire [31:0] dm_wdata;
wire [31:0] dm_rdata;

assign dm_wen   = {4{inst_SW}} & {4{resetn}}; // 内存写使能, 非 resetn 状态下有效
assign dm_addr  = alu_result;                 // 内存写地址, 为 ALU 结果
assign dm_wdata = rt_value;                   // 内存写数据, 为 rt 寄存器值

data_ram data_ram_module(
    .clk       (clk),          // I, 1, 时钟
    .wen       (dm_wen),       // I, 4, 写使能
    .addr      (dm_addr[6:2]), // I, 5, 读地址
    .wdata     (dm_wdata),     // I, 32, 写数据
    .rdata     (dm_rdata),     // O, 32, 读数据

    // display mem
    .test_addr (mem_addr[6:2]),
    .test_data (mem_data)
);
//----------------------------{访存}end-------------------------//


//----------------------------{写回}begin-----------------------//
wire inst_wdest_rt; // 寄存器堆写入地址为 rt 的指令
wire inst_wdest_rd; // 寄存器堆写入地址为 rd 的指令

```

```
assign inst_wdest_rt = inst_ADDIU | inst_LW | inst_LUI;
assign inst_wdest_rd = inst_ADDU | inst_SUBU | inst_SUB | inst_SLT | inst_AND | inst_NOR //
    【修改】新增inst_SUB，sub为R型指令，写回rd
                     | inst_OR | inst_XOR | inst_SLL | inst_SRL;

// 寄存器堆写使能信号，非复位状态下有效
assign rf_wen   = (inst_wdest_rt | inst_wdest_rd) & resetn;
assign rf_waddr = inst_wdest_rd ? rd : rt;          // 寄存器堆写地址 rd 或 rt
assign rf_wdata = inst_LW ? dm_rdata : alu_result;  // 写回结果
//--------------------------{写回}end-------------------------//

endmodule
```

A.2 inst_rom.v

Listing 2: inst_rom.v

```
`timescale 1ns / 1ps

//*************************************************************
// > 文件名: inst_rom.v
// > 描述   : 异步指令存储器模块，采用寄存器搭建而成，类似寄存器堆
// >          内嵌好指令，只读，异步读
// > 作者   : LOONGSON
// > 日期   : 2016-04-14
//*************************************************************

module inst_rom(
    input  [4 :0] addr, // 指令地址
    output reg [31:0] inst // 指令
);

wire [31:0] inst_rom[29:0]; // 【修改】原来为[19:0]，现在扩展到30条，用于加入Fib(10)程序

//------------- 指令编码 ---------|指令地址|--- 汇编指令 --|- 指令结果 -----//
assign inst_rom[ 0] = 32'h24010001; // 00H: addiu $1 ,$0,#1  | $1 = 0000_0001H
assign inst_rom[ 1] = 32'h00011100; // 04H: sll   $2 ,$1,#4  | $2 = 0000_0010H
assign inst_rom[ 2] = 32'h00411821; // 08H: addu  $3 ,$2,$1  | $3 = 0000_0011H
assign inst_rom[ 3] = 32'h00022082; // 0CH: srl   $4 ,$2,#2  | $4 = 0000_0004H
assign inst_rom[ 4] = 32'h00642823; // 10H: subu  $5 ,$3,$4  | $5 = 0000_000DH
assign inst_rom[ 5] = 32'hAC250013; // 14H: sw    $5 ,#19($1)| Mem[14H] = DH
assign inst_rom[ 6] = 32'h00A23027; // 18H: nor   $6 ,$5,$2  | $6 = FFFF_FFE2H
assign inst_rom[ 7] = 32'h00C33825; // 1CH: or    $7 ,$6,$3  | $7 = FFFF_FFF3H
assign inst_rom[ 8] = 32'h00E64026; // 20H: xor   $8 ,$7,$6  | $8 = 0000_0011H
assign inst_rom[ 9] = 32'hAC08001C; // 24H: sw    $8 ,#28($0)| Mem[1CH] = 11H
assign inst_rom[10] = 32'h00C7482A; // 28H: slt   $9 ,$6,$7  | $9 = 0000_0001H
assign inst_rom[11] = 32'h11210002; // 2CH: beq   $9 ,$1,#2  | 跳转到指令34H
assign inst_rom[12] = 32'h24010004; // 30H: addiu $1 ,$0,#4  | 不执行
```

```
32 assign inst_rom[13] = 32'h8C2A0013; // 34H: lw    $10,#19($1)| $10 = 0000_000DH
33 assign inst_rom[14] = 32'h15450003; // 38H: bne   $10,$5,#3  | 不跳转
34 assign inst_rom[15] = 32'h00415824; // 3CH: and   $11,$2,$1  | $11 = 0000_0000H
35 assign inst_rom[16] = 32'hAC0B001C; // 40H: sw    $11,#28($0)| Mem[1CH] = 0H
36 assign inst_rom[17] = 32'hAC040010; // 44H: sw    $4 ,#16($0)| Mem[10H] = 4H
37 assign inst_rom[18] = 32'h3C0C000C; // 48H: lui   $12,#12    | [R12] = C_0000H
38
39 //--------------------【新增】Fib(10)测试程序 begin--------------------//
40 // 说明：原00H-48H不动，在48H和原4CH的j 00H之间插入本段。
41 // 目标：计算Fib(10)=55=0x00000037。
42 // 寄存器使用：
43 // $13 = a，初始Fib(0)=0
44 // $14 = b，初始Fib(1)=1，循环结束后保存Fib(10)
45 // $15 = counter，循环计数器，初值9
46 // $16 = temp，临时保存a+b
47 // $17 = 常数1，用于sub递减counter
48 assign inst_rom[19] = 32'h240D0000; // 4CH: addiu $13,$0,#0    | 【新增】a = 0
49 assign inst_rom[20] = 32'h240E0001; // 50H: addiu $14,$0,#1    | 【新增】b = 1
50 assign inst_rom[21] = 32'h240F0009; // 54H: addiu $15,$0,#9    | 【新增】counter =
       9，循环9次得到Fib(10)
51
52 assign inst_rom[22] = 32'h01AE8021; // 58H: addu  $16,$13,$14  | 【新增】loop: temp = a + b
53 assign inst_rom[23] = 32'h01C06821; // 5CH: addu  $13,$14,$0   | 【新增】a = b
54 assign inst_rom[24] = 32'h02007021; // 60H: addu  $14,$16,$0   | 【新增】b = temp
55 assign inst_rom[25] = 32'h24110001; // 64H: addiu $17,$0,#1    | 【新增】$17 = 1
56 assign inst_rom[26] = 32'h01F17822; // 68H: sub   $15,$15,$17  | 【新增】验证sub：counter =
       counter - 1
57 assign inst_rom[27] = 32'h1DE0FFFB; // 6CH: bgtz  $15,loop     |
       【修改】验证bgtz：若counter>0，跳回58H；本CPU分支目标=当前PC+offset<<2，所以offset=-5
58 assign inst_rom[28] = 32'hAC0E0020; // 70H: sw    $14,#32($0)  | 【新增】将Fib(10)=0x37存入Mem[20H]
59 assign inst_rom[29] = 32'h08000000; // 74H: j     00H          | 【修改】原4CH的j
       00H移到最后，保证循环运行
60 //--------------------【新增】Fib(10)测试程序 end----------------------//
61
62 // 读指令，取4字节
63 always @(*)
64 begin
65     case (addr)
66         5'd0 : inst <= inst_rom[0 ];
67         5'd1 : inst <= inst_rom[1 ];
68         5'd2 : inst <= inst_rom[2 ];
69         5'd3 : inst <= inst_rom[3 ];
70         5'd4 : inst <= inst_rom[4 ];
71         5'd5 : inst <= inst_rom[5 ];
72         5'd6 : inst <= inst_rom[6 ];
73         5'd7 : inst <= inst_rom[7 ];
74         5'd8 : inst <= inst_rom[8 ];
75         5'd9 : inst <= inst_rom[9 ];
76         5'd10: inst <= inst_rom[10];
```

```
77          5'd11: inst <= inst_rom[11];
78          5'd12: inst <= inst_rom[12];
79          5'd13: inst <= inst_rom[13];
80          5'd14: inst <= inst_rom[14];
81          5'd15: inst <= inst_rom[15];
82          5'd16: inst <= inst_rom[16];
83          5'd17: inst <= inst_rom[17];
84          5'd18: inst <= inst_rom[18];
85          5'd19: inst <= inst_rom[19];
86          5'd20: inst <= inst_rom[20]; // 【新增】读取Fib程序指令
87          5'd21: inst <= inst_rom[21]; // 【新增】读取Fib程序指令
88          5'd22: inst <= inst_rom[22]; // 【新增】读取Fib程序指令
89          5'd23: inst <= inst_rom[23]; // 【新增】读取Fib程序指令
90          5'd24: inst <= inst_rom[24]; // 【新增】读取Fib程序指令
91          5'd25: inst <= inst_rom[25]; // 【新增】读取Fib程序指令
92          5'd26: inst <= inst_rom[26]; // 【新增】读取sub测试指令
93          5'd27: inst <= inst_rom[27]; // 【新增】读取bgtz测试指令
94          5'd28: inst <= inst_rom[28]; // 【新增】读取Fib结果写内存指令
95          5'd29: inst <= inst_rom[29]; // 【新增】读取跳回00H指令
96          default: inst <= 32'd0;
97      endcase
98  end
99
100 endmodule
```

A.3 tb.v

Listing 3: tb.v

```
1   `timescale 1ns / 1ps
2
3   ////////////////////////////////////////////////////////////////////////////////
4   // Company:
5   // Engineer:
6   //
7   // Create Date:   16:49:44 04/19/2016
8   // Design Name:   single_cycle_cpu
9   // Module Name:   F:/new_lab/6_single_cycle_cpu/tb.v
10  // Project Name:  single_cycle_cpu
11  // Target Device:
12  // Tool versions:
13  // Description:
14  //
15  // Verilog Test Fixture created by ISE for module: single_cycle_cpu
16  //
17  // Dependencies:
18  //
19  // Revision:
```

```
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////

module tb;

    // Inputs
    reg clk;
    reg resetn;
    reg [4:0] rf_addr;
    reg [31:0] mem_addr;

    // Outputs
    wire [31:0] rf_data;
    wire [31:0] mem_data;
    wire [31:0] cpu_pc;
    wire [31:0] cpu_inst;

    // Instantiate the Unit Under Test (UUT)
    single_cycle_cpu uut (
        .clk(clk),
        .resetn(resetn),
        .rf_addr(rf_addr),
        .mem_addr(mem_addr),
        .rf_data(rf_data),
        .mem_data(mem_data),
        .cpu_pc(cpu_pc),
        .cpu_inst(cpu_inst)
    );

    initial begin
        // Initialize Inputs
        clk = 0;
        resetn = 0;

        rf_addr = 5'd14;              // 【修改】观察寄存器$14, Fib(10)最后保存在$14中
        mem_addr = 32'h00000020;      // 【修改】观察Mem[20H], Fib(10)会被sw写到该地址

        // Wait 100 ns for global reset to finish
        #100;
        resetn = 1;

        // Add stimulus here
    end

    always #5 clk = ~clk;

endmodule
```